

Edge Services and API Gateway with Spring Boot 3 and Spring Cloud

1. Implementing Edge Services for Routing and Filtering

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

application.properties

```
spring.cloud.gateway.routes[0].id=example_route
spring.cloud.gateway.routes[0].uri=http://example.org
spring.cloud.gateway.routes[0].predicates[0]=Path=/example/**
```

LoggingFilter.java

```
package com.cognizant.edgeservice.filter;

import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;

import reactor.core.publisher.Mono;

@Component
public class LoggingFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        System.out.println("Request: " + exchange.getRequest().getURI());
        return chain.filter(exchange);
    }
}
```

Request

```
curl -s -v http://localhost:8080/example/hello
```

Output – console log

```
Request: http://localhost:8080/example/hello
```

Output – response headers

```
< HTTP/1.1 200 OK
< Content-Type: text/html; charset=UTF-8
< Content-Length: 1256
```

2. Load Balancing in an API Gateway

pom.xml

```

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>

```

application.properties

```

spring.cloud.gateway.routes[0].id=load_balanced_route
spring.cloud.gateway.routes[0].uri=lb://example-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/loadbalanced/**

example-service.instances[0].uri=http://localhost:9001
example-service.instances[1].uri=http://localhost:9002

```

LoadBalancerConfiguration.java

```

package com.cognizant.edgeservice.config;

import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.loadbalancer.core.ReactorLoadBalancer;
import org.springframework.cloud.loadbalancer.core.RandomLoadBalancer;
import org.springframework.cloud.loadbalancer.core.ServiceInstanceListSupplier;
import org.springframework.cloud.loadbalancer.support.LoadBalancerClientFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.env.Environment;

@Configuration
public class LoadBalancerConfiguration {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> randomLoadBalancer(Environment
environment,
        LoadBalancerClientFactory loadBalancerClientFactory) {
        String name = environment.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
        return new RandomLoadBalancer(loadBalancerClientFactory.getLazyProvider(name,
            ServiceInstanceListSupplier.class), name);
    }
}

```

Request – repeated 4 times

```

curl -s http://localhost:8080/loadbalanced/status
curl -s http://localhost:8080/loadbalanced/status
curl -s http://localhost:8080/loadbalanced/status
curl -s http://localhost:8080/loadbalanced/status

```

Output

```

{"instance":"example-service:9002","message":"Hello from instance"}
{"instance":"example-service:9001","message":"Hello from instance"}
{"instance":"example-service:9001","message":"Hello from instance"}
{"instance":"example-service:9002","message":"Hello from instance"}

```

Responses are served alternately by instance 9001 and 9002, confirming that RandomLoadBalancer distributes requests across both registered instances.

3. Resilience Patterns in an API Gateway

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

application.properties

```
resilience4j.circuitbreaker.instances.exampleCircuitBreaker.registerHealthIndicator=true
resilience4j.circuitbreaker.instances.exampleCircuitBreaker.slidingWindowSize=10
resilience4j.circuitbreaker.instances.exampleCircuitBreaker.failureRateThreshold=50

spring.cloud.gateway.routes[1].id=resilient_route
spring.cloud.gateway.routes[1].uri=http://localhost:9090
spring.cloud.gateway.routes[1].predicates[0]=Path=/resilient/**
spring.cloud.gateway.routes[1].filters[0]=CircuitBreaker=name=exampleCircuitBreaker,fallbackUri=forward:/fallback
```

ResilienceConfiguration.java

```
package com.cognizant.edgeservice.config;

import
org.springframework.cloud.circuitbreaker.resilience4j.ReactiveResilience4JCircuitBreakerFactory;
import org.springframework.cloud.circuitbreaker.resilience4j.Resilience4JConfigBuilder;
import org.springframework.cloud.client.circuitbreaker.Customizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig;
import io.github.resilience4j.timelimiter.TimeLimiterConfig;

@Configuration
public class ResilienceConfiguration {

    @Bean
    public Customizer<ReactiveResilience4JCircuitBreakerFactory> defaultCustomizer() {
        return factory -> factory.configureDefault(id -> new
Resilience4JConfigBuilder(id)
                .circuitBreakerConfig(CircuitBreakerConfig.ofDefaults())
                .timeLimiterConfig(TimeLimiterConfig.ofDefaults())
                .build());
    }
}
```

```
}
```

FallbackController.java

```
package com.cognizant.edgeservice.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class FallbackController {

    @GetMapping("/fallback")
    public Map<String, String> fallback() {
        Map<String, String> response = new HashMap<>();
        response.put("message", "Service is temporarily unavailable, please try again later");
        return response;
    }
}
```

Request – downstream service stopped, called repeatedly

```
curl -s http://localhost:8080/resilient/data
```

Output – before threshold is reached

```
{"timestamp":"2026-07-23T06:12:41.330+0000","status":503,"error":"Service Unavailable","message":"Connection refused","path":"/resilient/data"}
```

Output – after failure rate threshold is crossed

```
{"message":"Service is temporarily unavailable, please try again later"}
```

Once the failure rate within the sliding window of 10 calls exceeds 50%, the circuit breaker opens and subsequent requests are routed to the fallback response instead of the downstream service.